🚀 Hackathon 3.0: “Vision-to-Cart: The MCP Commerce Intelligence Challenge”

🎯 Objective

The objective of this hackathon is to push teams beyond traditional eCommerce implementations and build an **intelligent, AI-powered ordering system using MCP (Model Context Protocol) client-server architecture**.

Participants must design a system that can:

- Understand **visual inputs (images)** and map them to Magrabi products.
- Interpret **natural language intent** and convert it into precise product discovery.
- Seamlessly orchestrate multiple services using **MCP architecture**.
- Deliver a **high-quality, engaging UX** despite potentially long processing times.

The focus is not just on building features, but on demonstrating:

- Deep product understanding
- Strong system design thinking
- Intelligent orchestration of services
- Creative problem-solving under constraints

🧩 Problem Statement

You are tasked with building an **MCP-based intelligent ordering assistant** for Magrabi’s eCommerce platform.

This assistant should act as a **smart shopping companion** that can understand user intent from **images and natural language**, and guide them all the way to the Product Detail Page (PDP) and cart.

⚙️ Core Architecture Requirement (MANDATORY)

You must implement a **logical MCP (Model Context Protocol) client-server architecture**, which includes:

🔷 MCP Client

- Handles user interaction (image upload, chat input).
- Sends structured requests to MCP server.
- Displays progressive responses (not just final output).

🔷 MCP Server

- Acts as the **orchestrator**.

- Breaks down user intent into tasks.
- Calls appropriate tools/services such as:
    - Image recognition service
    - Semantic search service
    - Product database service
    - External web search API
- Maintains **context across the session**.

🔷 **Tools (Services)**

Each capability should be modular and callable:

- Product Search Tool
- Image Understanding Tool
- Web Search Tool
- Recommendation Engine

👉 **Hint:** Think of MCP Server as a "brain" coordinating multiple "specialized agents".

🧪 **Part 1: Image-Based Product Discovery**

🎯 **Goal**

Build a system where a user uploads any image, and the system identifies and matches it with semantic data of Magrabi products.

📌 **Requirements (Detailed)**

- The system must accept **any arbitrary image input**, not just catalog images.
- The uploaded image may contain:
    - Multiple objects
    - Background noise
    - Partial visibility of the product
- The system must:

1. Detect relevant objects (e.g., sunglasses, eyeglasses, frames).
2. Extract meaningful features such as:
    - Shape (aviator, round, square)
    - Frame type

        - Lens color
        - Brand cues (if possible)

3. Convert extracted features into a **searchable query representation**.
4. Perform **intelligent matching** against the product database.
5. Return:
        - Top matching products
        - Confidence score
6. Navigate to the most relevant **Product Detail Page (PDP)**.

🧠 **Critical Thinking Expectations**

- You must NOT rely on exact image matching.
- Your algorithm should handle:
    - "Similar looking" products
    - Incomplete or imperfect inputs

👉 **Hint Ideas:**

- Use embeddings (CLIP, OpenAI vision models, etc.)
- Hybrid approach: visual + metadata filtering
- Ranking algorithm (cosine similarity + heuristics)

💬 **Part 2: Natural Language to Product Mapping**

🎯 **Goal**

Enable users to search products using conversational queries.

📌 **Example Input**

"I want to buy the sunglasses worn by Akshay Khanna in Dhurandhar"

📌 **Requirements (Detailed)**

- The system must:
    1. Parse user intent from natural language.
    2. Identify:
        - Entity (celebrity: Akshay Khanna)

        - Context (movie: Dhurandhar)
        - Product type (sunglasses)
    3. Use a **real-time web search API** to:
        - Find references/images/articles
        - Extract visual/product clues
    4. Convert findings into structured product attributes.
    5. Search Magrabi catalog using derived attributes.
    6. Return best matching products and navigate to PDP.

🧠 **Critical Thinking Expectations**

- System should handle vague queries like:
    - "Something like what celebs wear at Cannes"
    - "Minimalist gold frame glasses"

👉 **Hint Ideas:**

- Use Tavily / SerpAPI for web search
- LLM for query decomposition
- Build an intermediate "intent schema"

🎨 **UX Challenge (HIGH IMPACT AREA)**

Since the process may take time, you must design a UX that:

- Keeps the user engaged during processing
- Shows **progressive feedback**, such as:
    - "Analyzing your image…"
    - "Finding similar styles…"
    - "Matching with catalog…"
- Enhances perceived performance using:
    - Skeleton loaders
    - Step-by-step progress indicators
    - Micro-interactions

👉 **Bonus Points for:**

- Streaming responses

- Conversational UI
- Visual explanation of matches

⏱️ **Time Boxing Strategy (8 Hours)**

Time slot	Activity
0 – 1 hr	Problem understanding + architecture design (MCP flow diagram)
1 – 3 hr	Implement MCP server + tool interfaces
3 – 5 hr	Build Part 1 (Image processing + matching logic)
5 – 6.5 hr	Build Part 2 (NL + web search integration)
6.5 – 7.5 hr	UX improvements + integration
7.5 – 8 hr	Testing + recording demo video

👉 **Strict Advice:**
Do NOT over-engineer early. Build a working pipeline first, then improve intelligence.

📊 **Evaluation Criteria (100 Marks)**

🔷 **1. MCP Architecture Implementation (20 Marks)**

- Clear separation of client, server, tools
- Proper orchestration and context handling

🔷 **2. Image Matching Intelligence (20 Marks)**

- Ability to handle non-exact images
- Smart ranking and similarity logic

🔷 **3. Natural Language Understanding (15 Marks)**

- Accurate intent parsing
- Effective use of web search + reasoning

🔷 **4. Product Mapping Accuracy (15 Marks)**

- Relevance of results
- Correct PDP navigation

🔷 **5. UX & User Engagement (15 Marks)**

- Handling of delays
- Feedback and interactivity

🔷 6. Code Quality & Modularity (10 Marks)

- Clean architecture
- Reusable components

🔷 7. Demo Clarity (5 Marks)

- Clear, concise, impactful video

🌟 Bonus Marks (Up to +10)

- Real-time streaming responses (+3)
- Multi-image understanding (+2)
- Personalized recommendations (+2)
- Innovative UX (voice, animation, etc.) (+3)

📦 Submission Guidelines

1. Create a **SHORT screen recording video (max 15 minutes)** demonstrating:
    - Image upload → product match → PDP → add to cart
    - Natural language query → product discovery
2. Submit **GitHub Repository** (using Kombee ID) including:
    - Complete source code
    - inputs/ folder (test data)
    - Demo video (MANDATORY)
3. Submit to your **Department Lead before 12th April, 12 AM**

🤖 Evaluation Note

- AI tools will be used to evaluate:
    - Code quality
    - Architecture patterns
    - Demo completeness

👉 Ensure your video is:

- Concise
- Covers ALL key flows
- Demonstrates real intelligence (not hardcoded logic)

🧠 Final Advice

This hackathon is not about building “just another feature.”

It’s about thinking like:

- A **systems architect**
- An **AI product designer**
- A **problem solver under ambiguity**

If your solution feels “too simple,” it probably is.

Push deeper. Think smarter. Build boldly.